

SMART HOME

Ahmed Mohamed , Abdullah , Abdelrahman

August 3, 2026

Abstract

An overview of our latest project !

1 Introduction

Smart homes are all about making daily tasks easy enough to handle with a simple voice command or gesture. Our project discusses a method to integrate simple activities with an Arduino and Machine learning .

2 Methodology

2.1 Data processing

The **Data Preprocessing** pipeline is responsible for transforming raw, unstructured audio signals into structured mathematical representations suitable for machine learning algorithms. Built upon **Clean (Hexagonal) Architecture**, the preprocessing system decouples abstract feature extraction contracts from concrete library implementations, ensuring modularity, scalability, and ease of maintenance.

2.1.1 Architectural Design & Separation of Concerns

To maintain a strict separation of concerns, the data preprocessing workflow relies on three primary layers:

- **Ports Layer (src/ports/)**: Defines the high-level abstract interfaces.
 - `feature_extractor.py`: Establishes the standard contract for converting raw audio signals into numerical feature matrices (e.g., MFCCs, Spectrograms).
 - `repository.py`: Defines abstract data-loading protocols for accessing datasets stored in `data/`.
- **Infrastructure Layer (src/infrastructure/)**: Implements concrete processing tools using external domain libraries.

- **librosa/**: Utilizes the `librosa` Python library to handle signal processing tasks such as Short-Time Fourier Transforms (STFT), Mel-Frequency Cepstral Coefficient (MFCC) extraction, zero-crossing rate computation, and audio trimming/normalization.
- **filesystem/**: Manages reading raw `.wav` or audio files from disk and outputting processed representations.
- **Application Layer (src/application/)**: Orchestrates data pipelines.
 - **pipeline.py**: Connects raw data ingestion, feature extraction, and feature scaling/transformation into an automated, reproducible sequential flow.

2.1.2 Data Preprocessing Workflow

The end-to-end transformation process follows a structured pipeline from raw audio ingestion to model input preparation, as illustrated in Figure 1.

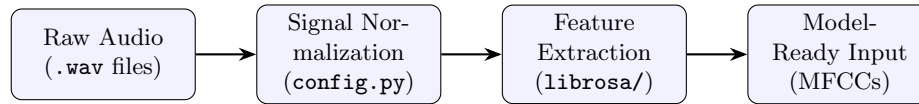


Figure 1: Data Preprocessing Ingestion and Feature Extraction Workflow.

1. Audio Ingestion & Data Management:

Raw audio recordings are ingested from the `data/` directory or captured dynamically via `recorder_app.py`. The `inventory.py` utility scans, categorizes, and logs available datasets to ensure proper dataset indexing.

2. Signal Standardization (config.py):

Before feature extraction, raw signals are standardized using globally configured parameters:

- **Resampling**: Unifying audio sampling rates across all input files.
- **Channel Conversion**: Converting multi-channel (stereo) audio to mono-channel signals.
- **Trimming & Padding**: Removing silent frames and standardizing clip durations.

3. Feature Extraction (src/infrastructure/librosa/):

The core mathematical transformation extracts time-frequency domain features depending on the downstream task:

- **Voice Command Recognition**: Extracts Mel-Frequency Cepstral Coefficients (MFCCs) to capture phonetic characteristics.

- **Speaker Identification:** Extracts pitch, spectral centroids, and spectral rolloff to capture speaker timbre and voice identity.
 - **Speech-to-Text (Transcription):** Prepares acoustic features compatible with the OpenAI Whisper pipeline.
4. **Pipeline Orchestration (`pipeline.py`):**
The extracted features are scaled, structured, and passed downstream to `trainer.py` or `evaluator.py` without exposing the internal signal processing logic to the model training scripts.

2.2 User interface

The user interface (UI) serves as the primary control center for the voice-controlled smart home system. Developed using the **Streamlit** framework, it offers a real-time web dashboard that interfaces directly with the hardware control scripts and underlying machine learning audio inference models.

2.2.1 Navigation & Sidebar Structure

The application utilizes a clean, persistent sidebar menu for switching between core operational views:

- **Dashboard:** Provides an overview of real-time system metrics, device states, and overall connection status.
- **Password:** Handles biometrics and voice password authentication.
- **Voice Control:** Manages real-time audio command capture, inference execution, and speaker classification display.
- **Settings:** Configures system parameters, thresholds, and operational preferences.

2.2.2 Core Application Modules

1. Dashboard View (Overview & System Monitoring) The main dashboard displays high-level status cards that summarize the real-time state of the connected hardware and system services:

- **Device State Indicators:** Dynamic display cards indicating the status of key peripherals:
 - *Lights Status:* Reflects current lighting state (ON / OFF).
 - *Music Status:* Shows audio output system status (ON / OFF).
 - *Voice Engine Status:* Displays current listening/processing state (e.g., Idle).
 - *AI Subsystem Status:* Shows readiness of downstream machine learning models (e.g., Ready).

- **Recent Activity & System Status:** Provides quick-glance diagnostics regarding current recognized user identity, latest executed command, and global health indicators (e.g., “*All Systems Running*”).

2. Authentication Module (Smart Home Password) Security and access control are managed via spoken passphrase verification:

- **Voice Password Entry:** Features a built-in microphone audio recorder widget enabling users to record password audio streams directly in the browser.
- **Access Gatekeeper:** Validates incoming voice samples before unlocking downstream control features or triggering feedback signals (e.g., hardware buzzer confirmation).

3. Voice Assistant Module (Interactive Voice Control) The primary execution interface for smart home voice control:

- **Live Command Capture:** An embedded real-time audio recorder that allows users to capture voice commands.
- **Inference Telemetry Cards:**
 - *Speaker Identification:* Displays recognized user identity computed by the speaker model.
 - *Confidence Metric:* Displays model prediction confidence percentage.
 - *Command Classification:* Shows parsed acoustic command (e.g., `LIGHT_ON`, `SEND_TEMP`).
 - *Engine Status:* Displays immediate execution state.
- **Action Log:** Summarizes the last executed command sent over the serial port to the microcontroller.

2.3 Hardware

Here comes the easiest part at all ! just an Arduino , two leds, a temperature sensor, and a Buzzer , all wired together as demonstrated in the following picture.

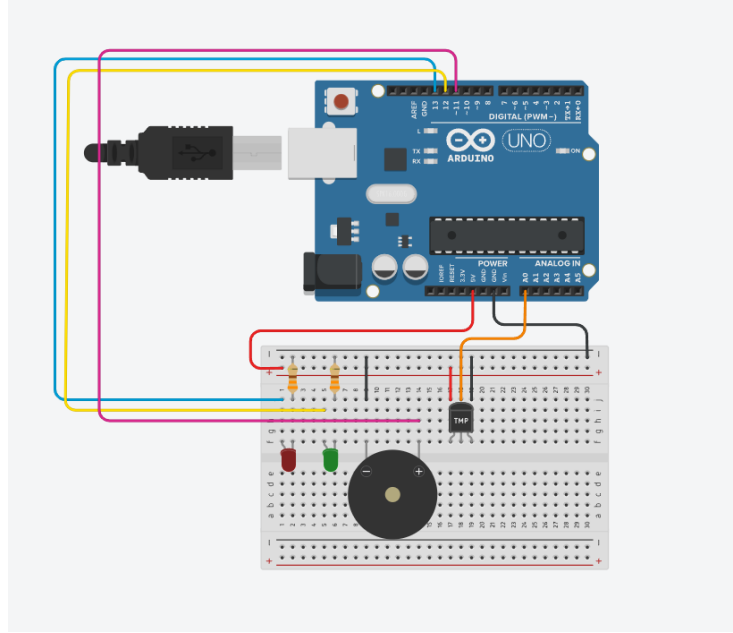


Figure 2: simulated on TinkerCad.

2.3.1 Authentication & Control Logic

The system's execution flow follows a sequential authentication model:

1. **User Inputs Password:** The user submits a voice password through the application interface.
2. **Validation Check:**
 - *Correct Password:* The Arduino triggers three buzzer alerts (confirming access), unlocking control permissions.
 - *Incorrect Password:* The system ignores the input and remains in a locked state.
3. **Command Execution:** Upon successful authentication and completion of the buzzer sequence, the user can perform two actions:
 - Turn the yellow LED light ON or OFF.
 - Turn ON music playback on the host device.

3 Pricing

Name	Arduino	Temperature Sensor	Buzzer
Ahmed Mohamed	NIL	59.5	6.5
Abdullah	NIL	NIL	NIL
Abdelrahman	NIL	NIL	NIL